Now an event and its setups are evaluated in the context of an Object that acts as a Clean Room. The instance variables in the setups and events are instance variables of the Clean Room rather than top-level instance variables. Because each event runs in its own Clean Room, events cannot share instance variables.

You might think of using a BasicObject instead of an Object for your Clean Room. However, remember that BasicObject is also a *Blank Slate (66)*, and as such it lacks some common methods, such as puts. So you should only use a BasicObject if you know that the code in the RedFlag events isn't going to call puts or other Object methods. You grin and add a comment to the code, leaving this difficult decision to Bill.

Wrap-Up

Here are a few spells and other interesting things that you learned today:

- What *Scope Gates (81)* are and how Ruby manages scope in general
- How to make bindings visible through scopes with *Flat Scopes (83)* and *Shared Scopes (84)*
- How to execute code in an object's scope (usually with instance_eval or instance_exec), or even in a *Clean Room (87)*
- How to turn a block into an object (a Proc) and back
- How to turn a method into an object (a Method or an UnboundMethod) and back
- What the differences are between the different types of callable objects: blocks, Procs, lambdas, and plain old methods
- How to write your own little DSL

That was a lot of new stuff in a single day. As you sneak out of the office, however, you can't shake the nagging feeling that you'll learn some of Ruby's best-kept secrets tomorrow.